PROGRAMMING IN JAVA

ASSIGNMENT 5 – CO5

DATA PERSISTENCE ENGINEERING – FILES AND DATABASES

Student Details	**To be completed**
Name	J.SUBHAASHREE
Register Number	192424008
Course / Section	AI&DS
Faculty	Ravi Pujitha
Submission Date	5.11.26
GitHub Repository Link	https://github.com/Subhaashree06/JAVA-Assignments/blob/main/CO5_Java_Assignment5_Code.ipynb

ASSIGNMENT OVERVIEW

Course Outcome CO5 focuses on Java input/output streams, file handling, object serialization and JDBC-based relational persistence. This report answers all five design tasks in Part A and all four implementation and experimental tasks in Part B. The report includes record layouts, pseudocode, complexity analysis, Java implementations, test cases, database design and engineering recommendations.

The engineering workflow followed is: Understand → Design → Protect → Implement → Measure → Challenge → Interpret.

Part	Task	Marks
A – Design	A1 File Format Design	7
A – Design	A2 Buffered I/O Design	5
A – Design	A3 Data-Access Pattern	5
A – Design	A4 Serialization Design	4
A – Design	A5 Transactions and Injection	4
B – Implementation	B1 Instrumented File Store	7
B – Implementation	B2 Adversarial Inputs	5
B – Implementation	B3 Statement vs PreparedStatement vs Batch	8
B – Implementation	B4 Final Engineering Decision	5
TOTAL		50

PART A – PERSISTENCE DESIGN AND PSEUDOCODE

TASK A1 – DESIGN A FILE FORMAT

A1.1 – Record Representation

The bus-pass record contains pass ID, student register number, name, route number, boarding stop, validity date and amount paid. For the implementation in this report, the compact five-field program record used in the supplied Complete-the-Code task is pass ID, register number, name, route number and amount. The full seven-field production layout below adds boarding stop and validity date.

Format	Field order and representation	Delimiter / size handling
Delimited text	passId, regNo, name, routeNo, boardingStop, validityDate, amount	UTF-8; comma-separated; quote or escape commas, quotes and newlines in text fields. Validate field count and numeric/date formats.
Binary DataOutputStream	writeUTF(passId), writeUTF(regNo), writeUTF(name), writeInt(routeNo), writeUTF(boardingStop), writeUTF(validityDate), writeDouble(amount)	Length-prefixed modified UTF strings; fixed-width int (4 bytes) and double (8 bytes). Add a version/magic header and record count.

A name containing a comma must not be split as an ordinary unescaped value. A CSV writer should quote the complete field and double embedded quote characters, or a simpler custom format can escape the delimiter and backslash. The reader must use the matching parser.

A1.2 – Operations and Stream Choices

Add: validate all fields, serialize one complete record, then append it with BufferedWriter and Files.newBufferedWriter using CREATE and APPEND. BufferedWriter reduces small write operations and try-with-resources closes the file safely.

Find by pass ID: use BufferedReader to read one line at a time, parse it, validate the expected number of fields and compare the pass ID. Stop when the matching record is found. If no record matches, return not found.

List by route: scan with BufferedReader, parse each valid row and collect rows whose route number equals the requested route. Damaged rows are skipped and counted or logged.

A1.2 – pseudocode

```
ALGORITHM AddPass(pass)
    VALIDATE pass fields
    OPEN file using BufferedWriter in APPEND mode
    WRITE encoded pass record
    WRITE newline
    CLOSE writer
END ALGORITHM

ALGORITHM FindPass(passID)
    IF file does not exist THEN RETURN NOT_FOUND
    OPEN file using BufferedReader
    WHILE line = READ_LINE()
        fields ← PARSE line
        IF field count is not 7 THEN CONTINUE
        IF fields are invalid THEN CONTINUE
        IF fields.passID = passID THEN
            CLOSE reader
            RETURN fields
        END IF
    END WHILE
    CLOSE reader
    RETURN NOT_FOUND
END ALGORITHM

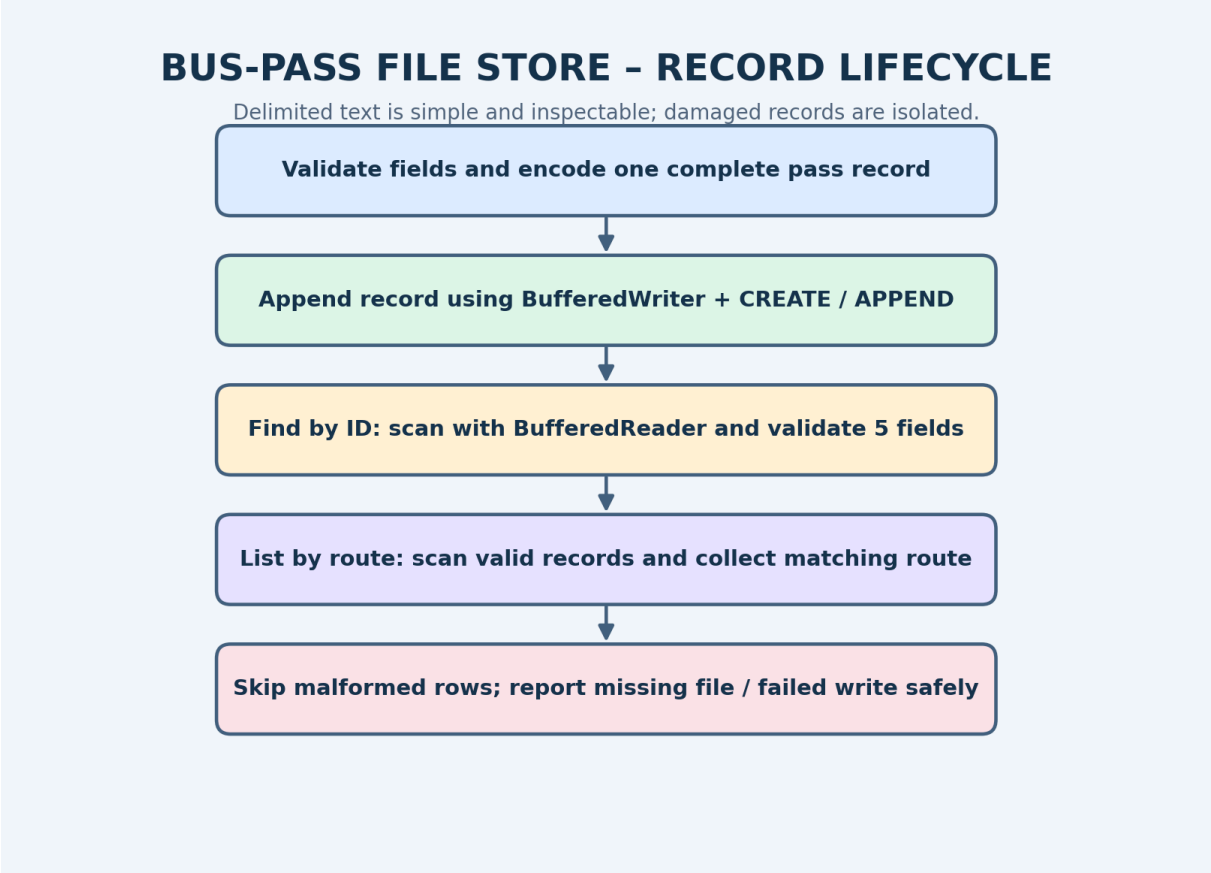
ALGORITHM ListPassesByRoute(routeNo)
    result ← empty list
    OPEN file using BufferedReader
    WHILE line = READ_LINE()
        fields ← PARSE line
        IF field count and field values are valid THEN
            IF fields.routeNo = routeNo THEN ADD fields TO result
        END IF
    END WHILE
    CLOSE reader
    RETURN result
END ALGORITHM
```

A1.3 – Damaged Data Handling

- Damaged line: verify field count, numeric route/amount and date format. Skip the invalid row, record its line number and continue.
- Missing file: check Files.exists(); for a search/list operation return an empty/not-found result or show a clear message. For adding, create the file if permitted.
- Incomplete write or power failure: write to a temporary file and atomically replace the original for full-file rewrites. For append operations, use a record checksum/length and detect a truncated final record on the next read. Keep backups and flush important writes.

Alternative Concept – RandomAccessFile and Complexity

With fixed-length records, each record occupies recordSize bytes. If records are indexed by position, record i is accessed by seek(i × recordSize), giving O(1) access by record number. Finding an arbitrary pass ID still requires O(n) scanning unless a separate index maps pass ID to record position. Fixed-width names and stops require padding; unused characters waste space. Text scanning is O(n) time and O(1) extra memory, while a direct positional lookup is O(1) time after the record index is known.



TASK A2 – DON'T READ WHAT YOU DON'T NEED

A2.1 – Predict the Cost

A 50 MB file contains approximately 52,428,800 bytes (if 1 MB = 1,048,576 bytes). A loop that calls FileInputStream.read() once per byte performs about 52.4 million Java-level read calls. Although the operating system and JVM may buffer internally, the repeated method calls, bounds checks and per-byte processing create avoidable overhead.

A2.2 – Buffered Redesign

Wrap FileInputStream in BufferedInputStream or read into a byte array such as 64 KB. Each block is scanned in memory and the buffer is refilled only when exhausted. For a 50 MiB file and a 64 KiB buffer, approximately 800 buffer refills are required instead of more than 52 million individual byte-level reads. The algorithm remains O(n) time and uses O(B) auxiliary memory, where B is buffer size.

```
ALGORITHM CountLinesBuffered(path)
    lineCount ← 0
    OPEN BufferedInputStream(path, 65536)
    WHILE byte ← READ() is not EOF
        IF byte = newline THEN lineCount ← lineCount + 1
```

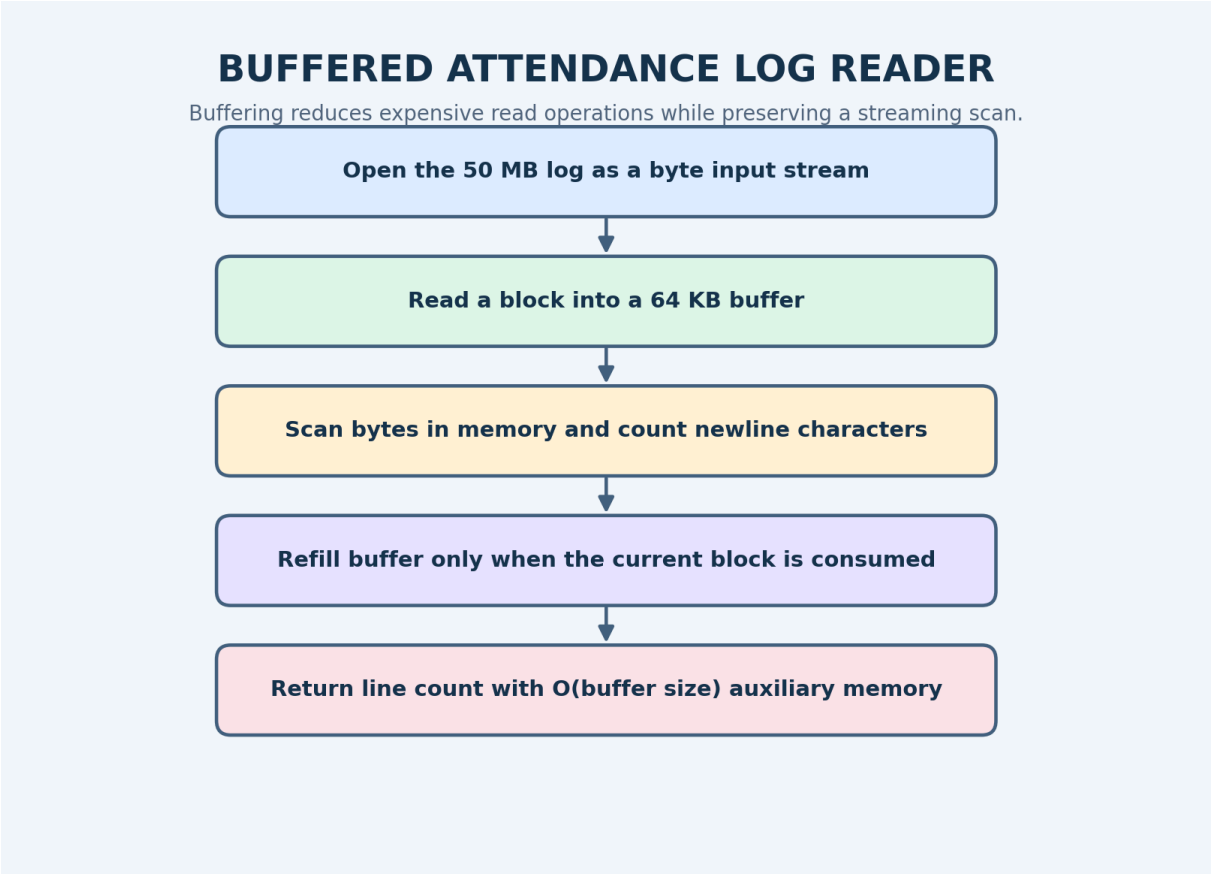
```
    END WHILE
    CLOSE stream
    RETURN lineCount
END ALGORITHM
```

A2.3 – When Buffering Does Not Help

Buffering may provide little benefit when the input is already held in memory, when the underlying API already supplies large blocks, or when the program performs only one tiny read. An extra buffer can add copying and memory overhead without reducing meaningful I/O.

Alternative – Files.readAllLines()

Files.readAllLines() loads all lines into a List<String>. Its time complexity is O(n), but memory use is O(n) in the file size plus Java String/List overhead. A buffered streaming reader uses O(B) memory and is preferable for a 50 MB or larger log when only a count is needed. readAllLines() is convenient for small files that must be processed repeatedly or randomly in memory.



TASK A3 – DISCOVER THE DATA-ACCESS DESIGN PATTERN

A3.1 – Common and Different Steps

StudentDB, CourseDB and MarksDB repeat connection acquisition, SQL execution, ResultSet traversal, object construction, exception handling and resource closing. Their differences are the entity type, table/column names, key type and SQL statements. The repeated behaviour belongs in a shared data-access pattern.

A3.2 – Generic DAO Interface

```
INTERFACE Dao<T, K>
    insert(item : T) : int
```

```
    findById(id : K) : T
    findAll() : List<T>
    update(item : T) : int
    delete(id : K) : int
END INTERFACE

CLASS StudentDao IMPLEMENTS Dao<Student, Integer>
    USE Connection supplied by constructor
    IMPLEMENT each method with PreparedStatement
    MAP ResultSet columns into Student objects
END CLASS
```

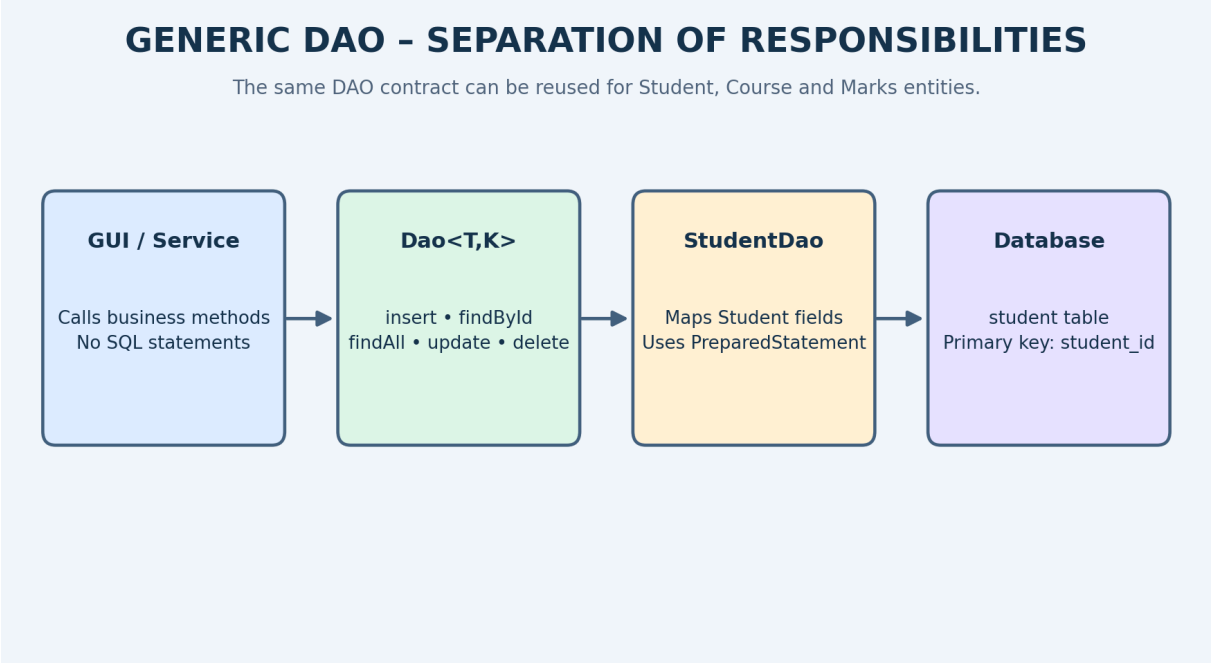
The supplied implementation provides Dao<T,K>, Student and StudentDao. StudentDao receives a Connection, uses parameterized SQL, maps database columns into Student fields and closes PreparedStatement/ResultSet resources with try-with-resources.

A3.3 – Object-to-Table Mapping

Student field	student table column	Mapping
studentId	student_id	INT PRIMARY KEY
registerNumber	reg_no	VARCHAR(20), UNIQUE, NOT NULL
name	name	VARCHAR(100), NOT NULL
email	email	VARCHAR(120), NOT NULL

Alternative – SQL in GUI or Service

If SQL is embedded directly in GUI/service classes, each class becomes coupled to schema details. Renaming a column may require editing many locations. With DAO, the SQL mapping is centralized in StudentDao and usually only that class changes. Both approaches have the same Big-O database complexity per query; DAO adds a small constant method-call/abstraction overhead but does not change asymptotic time or space.



TASK A4 – WHAT SHOULD BE SERIALIZED?

A4.1 – Decide What to Persist

Hostel, Room and Resident objects are part of the persistent object graph and should implement Serializable. The live database Connection must be transient because it represents an active external resource, not durable application data. Password should also be transient and preferably not stored in plaintext at all. If authentication information must persist, store a salted password hash in a secure database.

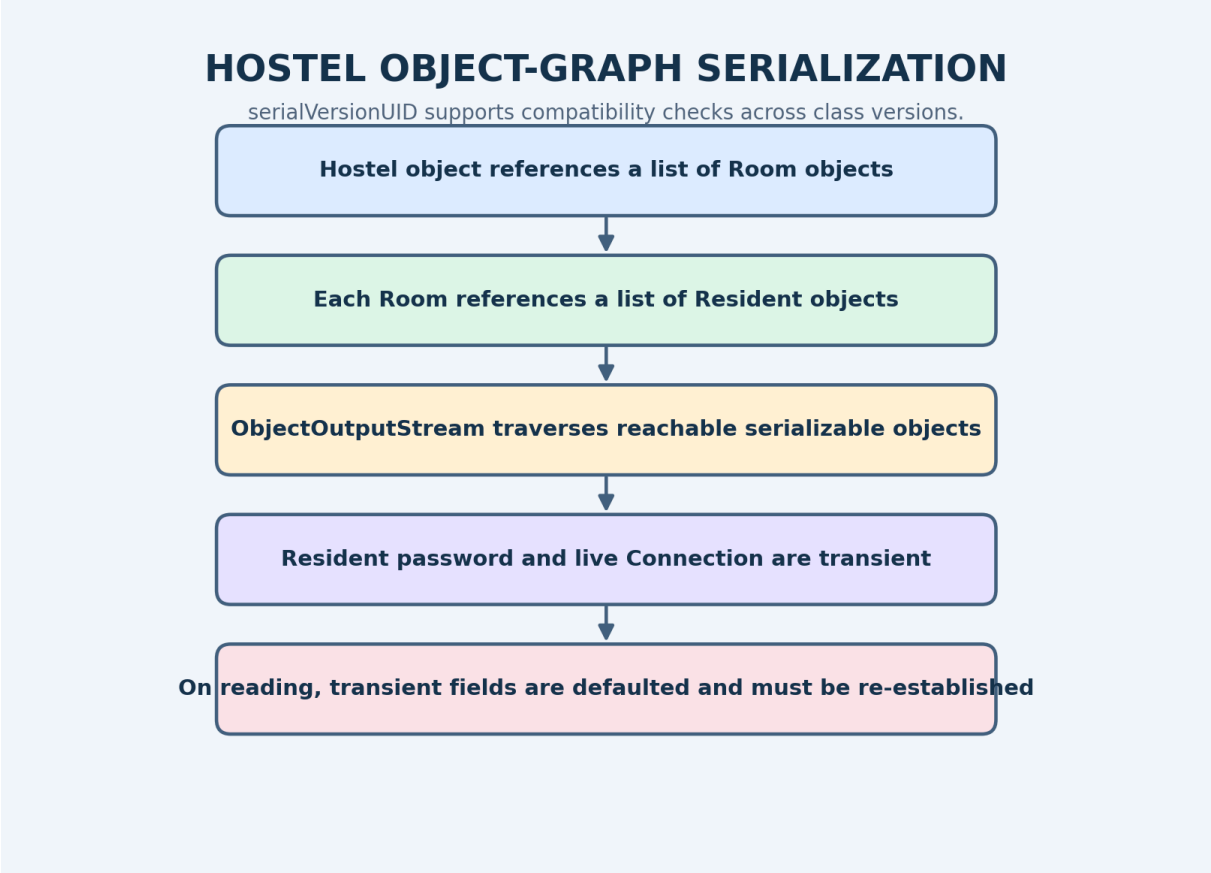
When Hostel is serialized, ObjectOutputStream follows references from Hostel to its Room list and from each Room to its Resident list. Every reachable non-transient serializable object is written as part of the object graph. Shared references are tracked so the same object is not duplicated indefinitely.

A4.2 – Plan for Change

When phoneNumber is added to Resident, an old serialized file can still be read if the class retains a compatible serialVersionUID. The new field receives its default value (null for String) when absent in the old stream. If the computed or declared serialVersionUID is incompatible, InvalidClassException may occur. Declare an explicit serialVersionUID and implement migration/defaulting logic for newly introduced fields.

Alternative – CSV or JSON

CSV/JSON is language-neutral, inspectable and supports explicit schema evolution, but requires parsing and validation. Java serialization is convenient for Java object graphs but is Java-specific and can be brittle across class changes. Both require O(n) time to write/read n residents; memory is O(n) if the entire graph/document is built in memory, or O(1)/O(buffer) extra memory with streaming JSON/CSV. Serialized binary data may be compact, while JSON is often larger due to field names and textual encoding.



TASK A5 – DESIGN FOR CORRECTNESS AND SAFETY

A5.1 – Transaction Design

A wallet transfer must be atomic. The debit and credit are executed in one transaction with auto-commit disabled. If either update fails, rollback restores both balances.

```
ALGORITHM Transfer(fromWallet, toWallet, amount)
    CHECK amount > 0
    SAVE previous autoCommit
    SET autoCommit = FALSE
    TRY
        UPDATE source wallet
            SET balance = balance - amount
            WHERE wallet_id = fromWallet AND balance >= amount
        IF affected rows != 1 THEN RAISE ERROR
        UPDATE destination wallet
            SET balance = balance + amount
            WHERE wallet_id = toWallet
        IF affected rows != 1 THEN RAISE ERROR
        COMMIT
    CATCH error
        ROLLBACK
        RAISE error
    FINALLY
        RESTORE previous autoCommit
END ALGORITHM
```

A5.2 – SQL Injection and PreparedStatement

The unsafe query concatenates user-controlled id and password into SQL. An input such as ' OR '1'='1 can change the WHERE expression so that it evaluates as true, potentially bypassing authentication. The exact outcome depends on the query and database contents.

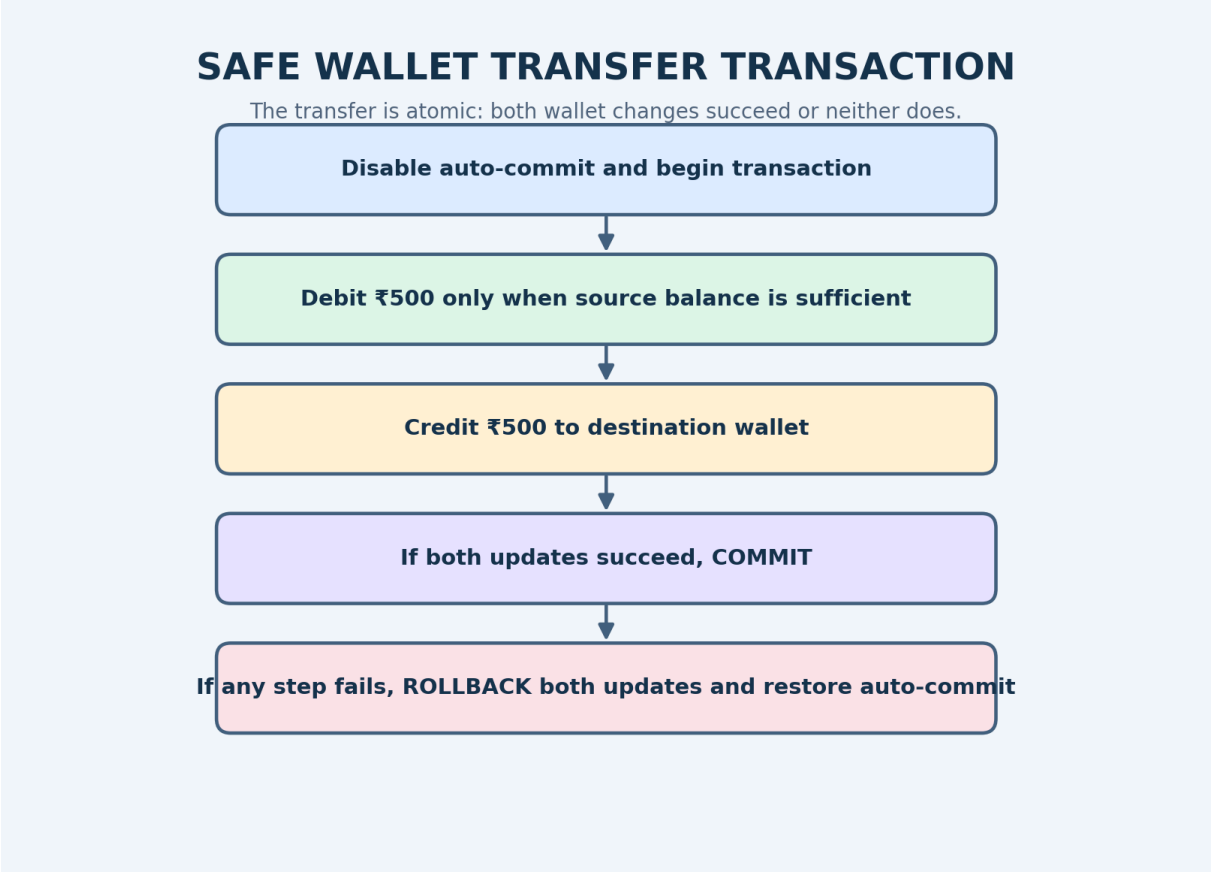
```
String sql = "SELECT COUNT(*) FROM app_user WHERE user_id=? AND user_pwd=?";
```

```
try (PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, id);
    ps.setString(2, password);
    try (ResultSet rs = ps.executeQuery()) {
        return rs.next() && rs.getInt(1) > 0;
    }
}
```

PreparedStatement sends parameter values separately from SQL syntax. The crafted input is treated as literal data, not executable SQL. Production systems should also store salted password hashes and use a password-verification function rather than compare plaintext passwords.

Alternative – Two Auto-Commit Updates

If debit and credit are separate auto-commit statements and the program fails after the debit, the source wallet loses ₹500 while the destination receives nothing. A transaction adds commit/rollback coordination and may add logging/locking overhead, but it protects correctness. The extra time is a justified constant-factor cost compared with the risk of inconsistent financial data.



PART B – IMPLEMENTATION AND EXPERIMENTAL INVESTIGATION

TASK B1 – IMPLEMENT AND INSTRUMENT THE FILE STORE

B1.1 – Validation Tests

PassStore implements append, find and route listing using buffered character streams and try-with-resources. It skips records that do not contain exactly five fields or contain invalid numeric route/amount values.

Test	Input / condition	Expected result	What it verifies
Normal find	P1003,192311003,Divya,7,3800	Returns P1003 fields	Valid record is found
Missing ID	Search P9999	null	Not-found handling
Damaged row	P1002,DAMAGED LINE	Skipped; search continues	Malformed row does not stop scan
Route listing	Route 7	Returns matching valid rows	Route filter works

B1.2 – Comparison with Pseudocode

The implementation follows the planned streaming scan and validation. The five-field Complete-the-Code version is retained to match the supplied TODO and expected output; the design section also documents the full seven-field production record. The parser uses split with a negative limit so trailing empty fields are preserved and can be rejected or validated.

B1.3 – Buffering Measurement

The notebook includes LineCounterBenchmark, which generates 1 MB, 10 MB and 50 MB test logs and times the unbuffered and 64 KB buffered methods. The program checks that both versions return the same line count. Exact elapsed times depend on the Colab runtime, file cache and storage environment; the measured values must be taken from the executed notebook rather than assumed.

File size	Unbuffered time (ms)	Buffered time (ms)	Line counts match
1 MB	Record from notebook run	Record from notebook run	Yes / verify
10 MB	Record from notebook run	Record from notebook run	Yes / verify
50 MB	Record from notebook run	Record from notebook run	Yes / verify

B1.4 – Measure Growth

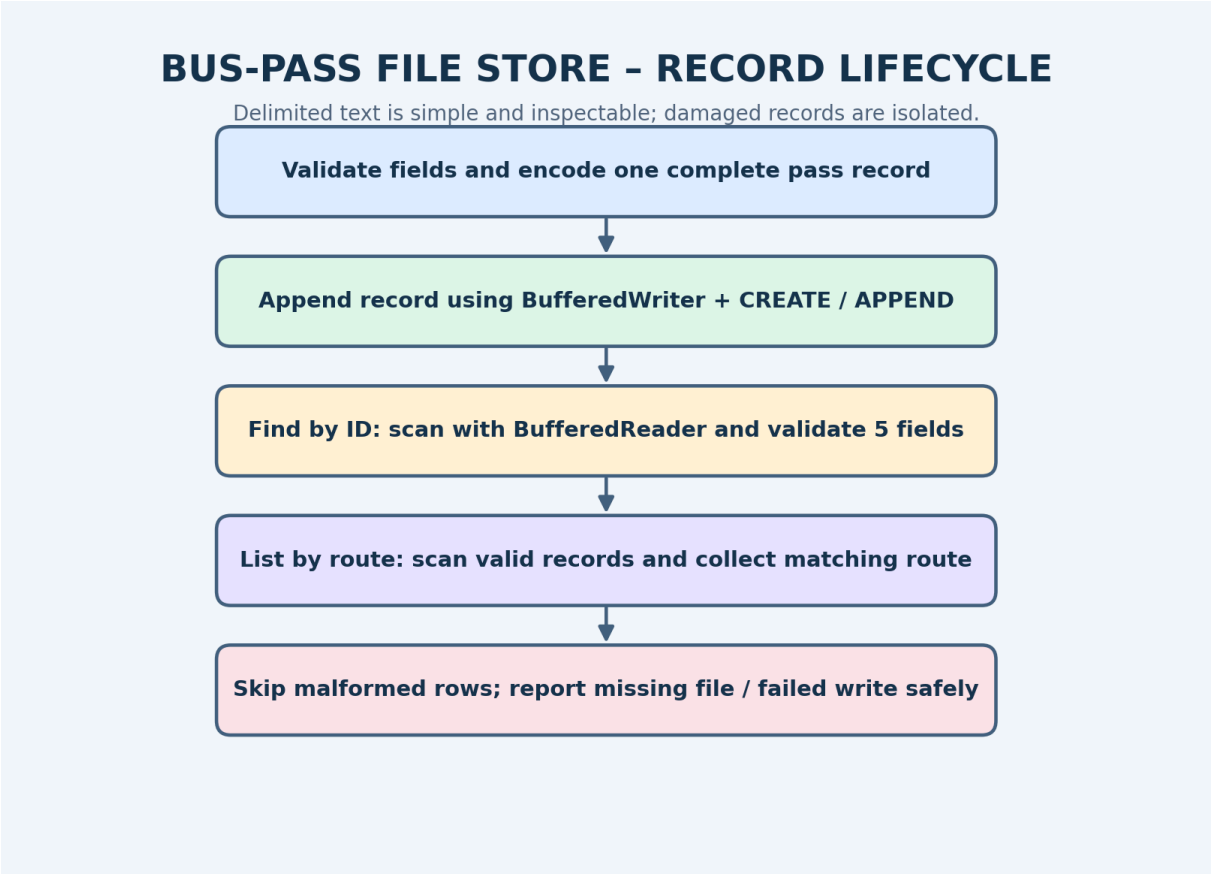
FileStoreExperiment generates 1,000, 10,000, 50,000 and 100,000 records and searches for the final ID. A linear scan has O(n) worst-case time because the target is at the end. The experiment prints elapsed milliseconds for each size; expected growth is approximately proportional to the number of records, subject to disk caching and runtime noise.

Records	Search time (ms)
1,000	Record from notebook run
10,000	Record from notebook run
50,000	Record from notebook run
100,000	Record from notebook run

Damaged P1002 does not stop the search because find() checks the field count and numeric fields, skips invalid records with continue, and proceeds to the next line. Thus a corrupt row is isolated rather than treated as end-of-file.

Alternative – HashMap Index

Load the file once into HashMap<String,String[]> keyed by pass ID. Building the map costs O(n) time and O(n) memory. Each average lookup is O(1), so q lookups cost O(n+q) expected time. Repeated file scans cost O(q×n) time but use O(1) auxiliary memory. A HashMap is appropriate when many searches are expected and memory is available; scanning is suitable for occasional queries and large files.



TASK B2 – ENGINEER AN ADVERSARIAL INPUT

Unsafe and Safe Login Tests

The notebook includes LoginSecurityDemo with an unsafe Statement-based query and a safe PreparedStatement-based query. The crafted password input ' OR '1'='1 is concatenated into the unsafe SQL and may turn the predicate into a tautology. With PreparedStatement it is bound as a literal password value, so the login is rejected unless that exact value is the stored password.

Input case	Unsafe Statement	PreparedStatement	Interpretation
Correct user and password	Accepts if credentials match	Accepts if credentials match	Normal authentication
Password: ' OR '1'='1	May bypass predicate	Rejects as literal value	SQL syntax injection blocked
Wrong number of CSV fields	Row may be misread	Not applicable	Field-count validation needed
Non-numeric amount	Parsing/database error	Rejected during validation/binding	Numeric validation needed

Empty line	Malformed record	Not applicable	Empty line is skipped/reported

Why Escaping or Regex Is Weaker

Manual quote escaping is easy to implement incorrectly and may differ across SQL dialects, character encodings and database modes. Regular expressions can validate a value's shape but cannot reliably separate SQL code from data in every context. PreparedStatement is the correct default because parameter values are not parsed as SQL syntax. Its binding overhead is generally small and the query remains O(1) setup plus database execution cost.

For the file-store adversarial tests, malformed rows should be logged with their line numbers and skipped. A production migration should write rejected rows to a failure report rather than silently discarding them.

TASK B3 – STATEMENT vs PREPAREDSTATEMENT vs BATCH

B3.1 – Correctness on Small Data

The SQL schema defines bus_pass with pass_id as the primary key, reg_no, name, route_no and amount. Ten rows can be inserted using each strategy, then selected and inspected with ResultSetMetaData. The metadata inspection program prints each column name, SQL type and display size.

Column	SQL type	Constraint
pass_id	VARCHAR(20)	PRIMARY KEY
reg_no	VARCHAR(20)	NOT NULL
name	VARCHAR(100)	NOT NULL
route_no	INT	NOT NULL
amount	DECIMAL(10,2)	NOT NULL

B3.2 – Data-Size Comparison

JdbcBenchmark contains Statement, PreparedStatement and PreparedStatement batch approaches. It tests 10, 1,000, 10,000 and 100,000 rows and records elapsed milliseconds. The required 1,000, 10,000 and 100,000 measurements should be captured from a connected MySQL instance. Typical behaviour is that Statement repeatedly parses SQL, PreparedStatement reuses a parameterized statement, and batching reduces client/server round trips.

Rows	Statement (ms)	PreparedStatement (ms)	Batch (ms)
1,000	Run on MySQL	Run on MySQL	Run on MySQL
10,000	Run on MySQL	Run on MySQL	Run on MySQL
100,000	Run on MySQL	Run on MySQL	Run on MySQL

B3.3 – Auto-Commit vs Transaction

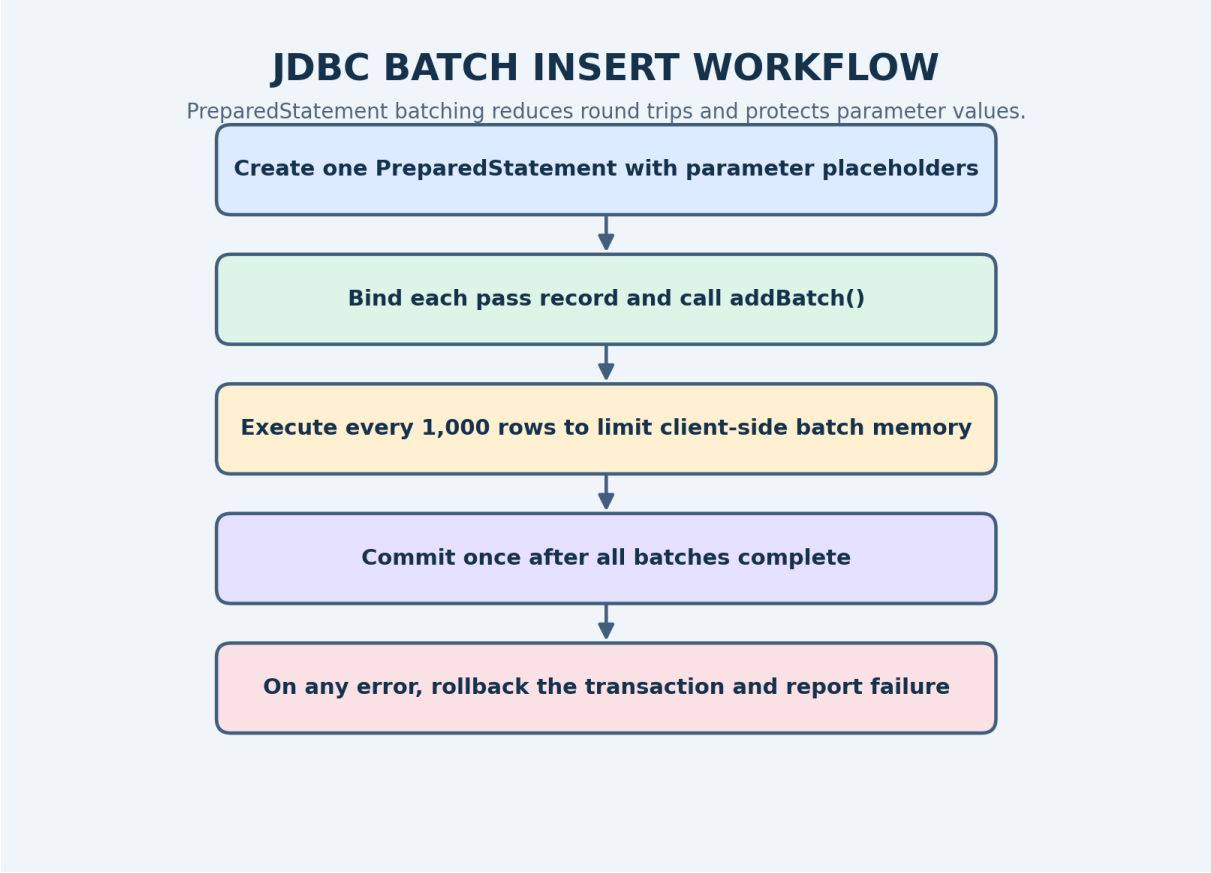
With auto-commit enabled, each successful insert is committed independently. If failure occurs halfway, earlier rows remain in the table. With one transaction, a failure triggers rollback and none of the transaction's inserted rows remain. The supplied PassLoader disables auto-commit, executes batches of 1,000, commits only after all rows succeed, and rolls back on SQLException or RuntimeException.

B3.4 – Engineering Trade-Off

- Nightly import of 100,000 passes: use PreparedStatement batches in a transaction, with bounded batch size, validation and a failure report.
- Single pass issued at the counter: use PreparedStatement for parameter safety and straightforward immediate feedback; batching is unnecessary for one row.

Alternative – Multi-row INSERT and LOAD DATA

A multi-row INSERT sends many values in one statement and can reduce round trips, but SQL statement size and parameter count grow with batch size. LOAD DATA is often fastest for large file imports when the server can access the file and permissions allow it. PreparedStatement batching uses O(b) client memory for batch size b; streaming a very large file into bounded batches keeps memory controlled. All methods have O(n) work overall, but round-trip and parsing constants differ.



TASK B4 – FINAL ENGINEERING DECISION

B4.1 – Practical Migration Strategy

PassMigration reads the existing CSV file line by line, validates field count, route number and amount, and writes invalid rows with their line number and reason to migration-failures.txt. Valid records are loaded through a batch PreparedStatement transaction. If a database insert fails, the transaction is rolled back.

B4.2 – Database Metadata

DatabaseInspector uses DatabaseMetaData to report the database product name and version, JDBC driver name and version, and the tables visible in the current catalog. Metadata is useful after

deployment to confirm the application is connected to the expected database and driver, and to identify environment differences during migration to another server.

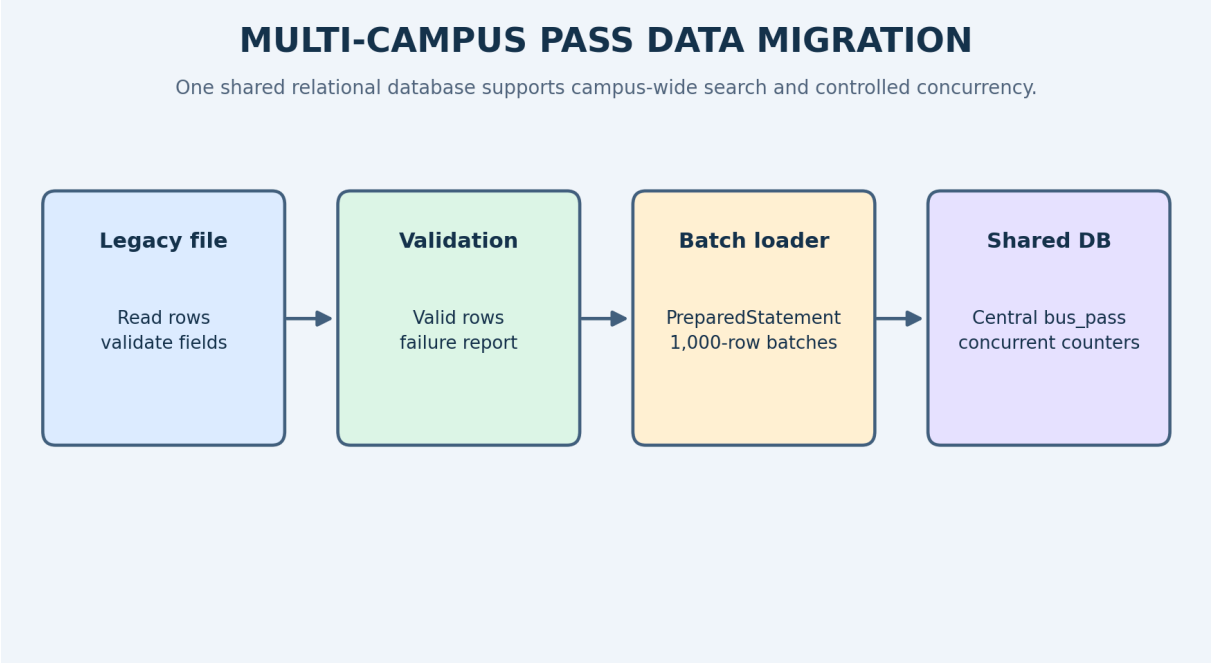
B4.3 – Production Decision

For 12 campuses and approximately 100 times the current number of passes, a shared relational database is the recommended production store. It provides indexed lookup by pass ID, route filtering, constraints, transactions and controlled concurrent access from multiple counters. Text files are useful for interchange, backups and audit exports but are weak as a shared concurrent transactional store. Java serialization is unsuitable as the primary multi-campus database because it is Java-specific and does not provide relational querying or multi-user transaction management.

Requirement	Text file	Serialization	Relational database
Concurrent counters	Requires external locking	Requires external locking	Transactions and concurrency control
Search by ID	O(n) scan or external index	Custom index required	Indexed lookup, typically O(log n)
Route queries	Full scan or custom index	Custom traversal/index	SQL index and query
Crash safety	Needs careful atomic writes/backups	Needs careful file replacement	Transaction log and recovery
Schema evolution	Manual parsing/versioning	serialVersionUID and migration	Schema migrations and constraints
12-campus sharing	Difficult	Difficult	Designed for centralized access

Alternative – One Serialized File per Campus

With 12 separate serialized files, a global search may require scanning all 12 files, O(N) total over all passes, and each campus must coordinate updates, backups and version changes. Storage remains O(N) overall, but duplicated application logic and synchronization are required. Concurrent counters at one campus would need file locks; cross-campus consistency and duplicate pass IDs are difficult. A shared database provides centralized constraints, transactions and indexes, so it is the better engineering choice.



FINAL CONCLUSION

This assignment demonstrates that persistence design must be selected according to data size, access patterns, safety and concurrency requirements. Buffered streams are appropriate for large sequential files; DAO separates SQL from business logic; serialization is useful for controlled Java object snapshots but requires transient-resource handling and versioning; JDBC PreparedStatement protects parameters; and transactions preserve correctness for multi-step updates. For the expanded multi-campus bus-pass system, a shared relational database with indexed queries, bounded batch imports, validation and transaction control is the recommended solution.

APPENDIX A – COMPLETE JAVA SOURCE CODE

The following source listings are extracted from the Assignment 5 Colab notebook. They are included so that the report and executable implementation remain aligned. JDBC programs require MySQL Connector/J and a configured database connection.

Task01_FileStore/PassStore.java

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardOpenOption;
import java.util.ArrayList;
import java.util.List;

public class PassStore {
    private final Path file;

    public PassStore(Path file) {
        this.file = file;
    }

    public void add(String passId, String regNo, String name, int route, double amount)
            throws IOException {
        try (BufferedWriter writer = Files.newBufferedWriter(
                file, StandardOpenOption.CREATE, StandardOpenOption.APPEND)) {
            writer.write(String.join(",",
                    clean(passId), clean(regNo), clean(name),
                    String.valueOf(route), String.valueOf(amount)));
            writer.newLine();
        }
    }

    private String clean(String value) {
        return value == null ? "" : value.replace(",", " ").replace("\n", " ").trim();
    }

    public String[] find(String passId) throws IOException {
        if (!Files.exists(file)) return null;
        try (BufferedReader reader = Files.newBufferedReader(file)) {
            String line;
            while ((line = reader.readLine()) != null) {
                String[] fields = line.split(",", -1);
                if (fields.length != 5) continue;
                try {
                    Integer.parseInt(fields[3].trim());
                    Double.parseDouble(fields[4].trim());
                } catch (NumberFormatException damaged) {
                    continue;
                }
                if (fields[0].trim().equals(passId)) return fields;
            }
        }
        return null;
    }

    public List<String[]> listByRoute(int route) throws IOException {
        List<String[]> matches = new ArrayList<>();
        if (!Files.exists(file)) return matches;
        try (BufferedReader reader = Files.newBufferedReader(file)) {
            String line;
            while ((line = reader.readLine()) != null) {
                String[] fields = line.split(",", -1);
                if (fields.length != 5) continue;
                try {
                    if (Integer.parseInt(fields[3].trim()) == route)
                        matches.add(fields);
                    Double.parseDouble(fields[4].trim());
                } catch (NumberFormatException ignored) {
                    // Ignore damaged records and continue scanning.
                }
            }
        }
        return matches;
    }

    public static void main(String[] args) throws Exception {
        Path path = Files.createTempFile("bus-passes", ".csv");
        PassStore store = new PassStore(path);
        store.add("P1001", "192311001", "Arun", 12, 4500);
        Files.writeString(path, "P1002,DAMAGED LINE\n", StandardOpenOption.APPEND);
```

```
        store.add("P1003", "192311003", "Divya", 7, 3800);
        System.out.println(java.util.Arrays.toString(store.find("P1003")));
        System.out.println(java.util.Arrays.toString(store.find("P1002")));
        System.out.println("Route 7 count: " + store.listByRoute(7).size());
        Files.deleteIfExists(path);
    }
}
```

Task01_FileStore/BufferedLineCounter.java

```
import java.io.BufferedInputStream;
import java.io.FileInputStream;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

public class BufferedLineCounter {
    public static long countUnbuffered(Path file) throws IOException {
        long lines = 0;
        boolean hasData = false;
        try (FileInputStream in = new FileInputStream(file.toFile())) {
            int value;
            while ((value = in.read()) != -1) {
                hasData = true;
                if (value == '\n') lines++;
            }
        }
        return lines + (hasData && !endsWithNewline(file) ? 1 : 0);
    }

    public static long countBuffered(Path file) throws IOException {
        long lines = 0;
        boolean hasData = false;
        try (BufferedInputStream in =
                    new BufferedInputStream(new FileInputStream(file.toFile()), 64 * 1024)) {
            int value;
            while ((value = in.read()) != -1) {
                hasData = true;
                if (value == '\n') lines++;
            }
        }
        return lines + (hasData && !endsWithNewline(file) ? 1 : 0);
    }

    private static boolean endsWithNewline(Path file) throws IOException {
        long size = Files.size(file);
        if (size == 0) return true;
        try (var channel = java.nio.channels.FileChannel.open(file)) {
            java.nio.ByteBuffer one = java.nio.ByteBuffer.allocate(1);
            channel.position(size - 1);
            channel.read(one);
            return one.array()[0] == '\n';
        }
    }

    public static void main(String[] args) throws Exception {
        Path file = Files.createTempFile("attendance", ".log");
        StringBuilder data = new StringBuilder();
        for (int i = 0; i < 100000; i++) data.append("ATTENDANCE-").append(i).append('\n');
        Files.writeString(file, data);
        long start = System.nanoTime();
        long a = countUnbuffered(file);
        long unbufferedMs = (System.nanoTime() - start) / 1_000_000;
        start = System.nanoTime();
        long b = countBuffered(file);
        long bufferedMs = (System.nanoTime() - start) / 1_000_000;
        System.out.println("Lines: " + a + " / " + b);
        System.out.println("Unbuffered ms: " + unbufferedMs);
        System.out.println("Buffered ms: " + bufferedMs);
        Files.deleteIfExists(file);
    }
}
```

Task02_DataAccess/Dao.java

```
import java.sql.SQLException;
import java.util.List;

public interface Dao<T, K> {
    int insert(T item) throws SQLException;
    T findById(K id) throws SQLException;
    List<T> findAll() throws SQLException;
    int update(T item) throws SQLException;
    int delete(K id) throws SQLException;
}
```

Task02_DataAccess/Student.java

```
public class Student {
    private final int studentId;
    private final String registerNumber;
    private final String name;
    private final String email;

    public Student(int studentId, String registerNumber, String name, String email) {
        this.studentId = studentId;
        this.registerNumber = registerNumber;
        this.name = name;
        this.email = email;
    }

    public int getStudentId() { return studentId; }
    public String getRegisterNumber() { return registerNumber; }
    public String getName() { return name; }
    public String getEmail() { return email; }

    @Override
    public String toString() {
        return studentId + " | " + registerNumber + " | " + name + " | " + email;
    }
}
```

Task02_DataAccess/StudentDao.java

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class StudentDao implements Dao<Student, Integer> {
    private final Connection connection;

    public StudentDao(Connection connection) {
        this.connection = connection;
    }

    @Override
    public int insert(Student s) throws SQLException {
        String sql = "INSERT INTO student(student_id, reg_no, name, email) VALUES (?, ?, ?, ?)";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setInt(1, s.getStudentId());
            ps.setString(2, s.getRegisterNumber());
            ps.setString(3, s.getName());
            ps.setString(4, s.getEmail());
            return ps.executeUpdate();
        }
    }

    @Override
    public Student findById(Integer id) throws SQLException {
        String sql = "SELECT student_id, reg_no, name, email FROM student WHERE student_id = ?";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setInt(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next() ? new Student(rs.getInt("student_id"),
                        rs.getString("reg_no"), rs.getString("name"), rs.getString("email")) : null;
            }
        }
    }

    @Override
    public List<Student> findAll() throws SQLException {
        List<Student> students = new ArrayList<>();
        try (Statement st = connection.createStatement();
             ResultSet rs = st.executeQuery(
                     "SELECT student_id, reg_no, name, email FROM student ORDER BY student_id")) {
            while (rs.next()) students.add(new Student(rs.getInt(1), rs.getString(2),
                    rs.getString(3), rs.getString(4)));
        }
        return students;
    }

    @Override
    public int update(Student s) throws SQLException {
        String sql = "UPDATE student SET reg_no=?, name=?, email=? WHERE student_id=?";
        try (PreparedStatement ps = connection.prepareStatement(sql)) {
            ps.setString(1, s.getRegisterNumber());
            ps.setString(2, s.getName());
            ps.setString(3, s.getEmail());
            ps.setInt(4, s.getStudentId());
            return ps.executeUpdate();
        }
    }

    @Override
    public int delete(Integer id) throws SQLException {
```

```
        try (PreparedStatement ps =
                    connection.prepareStatement("DELETE FROM student WHERE student_id=?")) {
            ps.setInt(1, id);
            return ps.executeUpdate();
        }
    }
}
```

Task03_Serialization/Hostel.java

```
import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;

public class Hostel implements Serializable {
    private static final long serialVersionUID = 1L;
    private final String hostelName;
    private final List<Room> rooms = new ArrayList<>();

    public Hostel(String hostelName) { this.hostelName = hostelName; }
    public void addRoom(Room room) { rooms.add(room); }
    public String getHostelName() { return hostelName; }
    public List<Room> getRooms() { return rooms; }
}
```

Task03_Serialization/Room.java

```
import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;

public class Room implements Serializable {
    private static final long serialVersionUID = 1L;
    private final int roomNumber;
    private final List<Resident> residents = new ArrayList<>();

    public Room(int roomNumber) { this.roomNumber = roomNumber; }
    public void addResident(Resident resident) { residents.add(resident); }
    public int getRoomNumber() { return roomNumber; }
    public List<Resident> getResidents() { return residents; }
}
```

Task03_Serialization/Resident.java

```
import java.io.Serializable;
import java.sql.Connection;

public class Resident implements Serializable {
    private static final long serialVersionUID = 1L;
    private final String name;
    private final String registerNumber;
    private final String phoneNumber;
    private transient String password;
    private transient Connection liveConnection;

    public Resident(String name, String registerNumber, String phoneNumber,
                    String password, Connection liveConnection) {
        this.name = name;
        this.registerNumber = registerNumber;
        this.phoneNumber = phoneNumber;
        this.password = password;
        this.liveConnection = liveConnection;
    }

    public String getName() { return name; }
    public String getRegisterNumber() { return registerNumber; }
    public String getPhoneNumber() { return phoneNumber; }
    public String getPassword() { return password; }
    public Connection getLiveConnection() { return liveConnection; }
}
```

Task03_Serialization/SerializationDemo.java

```
import java.io.*;

public class SerializationDemo {
    public static void main(String[] args) throws Exception {
        Hostel hostel = new Hostel("SIMATS Hostel");
        Room room = new Room(101);
        room.addResident(new Resident("Arun", "192311001", "9876543210",
                "not-persisted", null));
        hostel.addRoom(room);

        File file = new File("hostel-state.ser");
        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(file))) {
            out.writeObject(hostel);
        }
```

```
        try (ObjectInputStream in = new ObjectInputStream(new FileInputStream(file))) {
            Hostel restored = (Hostel) in.readObject();
            Resident resident = restored.getRooms().get(0).getResidents().get(0);
            System.out.println(restored.getHostelName());
            System.out.println(resident.getName() + " " + resident.getPhoneNumber());
            System.out.println("Password after load: " + resident.getPassword());
            System.out.println("Connection after load: " + resident.getLiveConnection());
        }
        file.delete();
    }
}
```

Task04_Transactions/WalletTransfer.java

```
import java.math.BigDecimal;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class WalletTransfer {
    public static void transfer(Connection con, int fromWallet, int toWallet,
                                BigDecimal amount) throws SQLException {
        if (amount == null || amount.signum() <= 0)
            throw new IllegalArgumentException("Amount must be positive");

        boolean oldAutoCommit = con.getAutoCommit();
        con.setAutoCommit(false);
        try {
            try (PreparedStatement debit = con.prepareStatement(
                    "UPDATE wallet SET balance = balance - ? WHERE wallet_id = ? AND balance >= ?");
                 PreparedStatement credit = con.prepareStatement(
                    "UPDATE wallet SET balance = balance + ? WHERE wallet_id = ?")) {
                debit.setBigDecimal(1, amount);
                debit.setInt(2, fromWallet);
                debit.setBigDecimal(3, amount);
                if (debit.executeUpdate() != 1)
                    throw new SQLException("Debit failed: wallet missing or insufficient balance");

                credit.setBigDecimal(1, amount);
                credit.setInt(2, toWallet);
                if (credit.executeUpdate() != 1)
                    throw new SQLException("Credit failed: destination wallet not found");
            }
            con.commit();
        } catch (SQLException | RuntimeException ex) {
            con.rollback();
            throw ex;
        } finally {
            con.setAutoCommit(oldAutoCommit);
        }
    }
}
```

Task04_Transactions/LoginSecurityDemo.java

```
import java.sql.*;

public class LoginSecurityDemo {
    public static boolean unsafeLogin(Connection con, String id, String password)
            throws SQLException {
        String sql = "SELECT COUNT(*) FROM app_user WHERE user_id='" + id
                + "' AND user_pwd='" + password + "'";
        try (Statement st = con.createStatement(); ResultSet rs = st.executeQuery(sql)) {
            return rs.next() && rs.getInt(1) > 0;
        }
    }

    public static boolean safeLogin(Connection con, String id, String password)
            throws SQLException {
        String sql = "SELECT COUNT(*) FROM app_user WHERE user_id=? AND user_pwd=?";
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, id);
            ps.setString(2, password);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next() && rs.getInt(1) > 0;
            }
        }
    }

    public static void main(String[] args) throws Exception {
        String url = System.getenv().getOrDefault("DB_URL",
                "jdbc:mysql://localhost:3306/assignment5");
        String user = System.getenv().getOrDefault("DB_USER", "root");
        String password = System.getenv().getOrDefault("DB_PASSWORD", "");
        try (Connection con = DriverManager.getConnection(url, user, password)) {
            String attack = "' OR '1'='1";
            System.out.println("Unsafe accepts crafted input: " +
```

```
                unsafeLogin(con, "student1", attack));
            System.out.println("PreparedStatement accepts crafted input: " +
                    safeLogin(con, "student1", attack));
        }
    }
}
```

Task05_JdbcBatch/PassLoader.java

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.List;

public class PassLoader {
    public static int loadAll(Connection con, List<String[]> passes) throws SQLException {
        String sql = "INSERT INTO bus_pass(pass_id, reg_no, name, route_no, amount) "
                + "VALUES (?, ?, ?, ?, ?)";
        boolean oldAutoCommit = con.getAutoCommit();
        con.setAutoCommit(false);
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            int count = 0;
            for (String[] pass : passes) {
                if (pass.length != 5) throw new SQLException("Pass must contain five fields");
                ps.setString(1, pass[0]);
                ps.setString(2, pass[1]);
                ps.setString(3, pass[2]);
                ps.setInt(4, Integer.parseInt(pass[3]));
                ps.setDouble(5, Double.parseDouble(pass[4]));
                ps.addBatch();
                count++;
                if (count % 1000 == 0) ps.executeBatch();
            }
            ps.executeBatch();
            con.commit();
            return count;
        } catch (SQLException | RuntimeException ex) {
            con.rollback();
            throw ex;
        } finally {
            con.setAutoCommit(oldAutoCommit);
        }
    }
}
```

Task05_JdbcBatch/JdbcBenchmark.java

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class JdbcBenchmark {
    private static List<String[]> makeRows(int count) {
        List<String[]> rows = new ArrayList<>();
        for (int i = 1; i <= count; i++)
            rows.add(new String[]{"P" + i, "REG" + i, "Student" + i,
                    String.valueOf((i % 20) + 1), String.valueOf(1000 + i)});
        return rows;
    }

    private static void clear(Connection con) throws SQLException {
        try (Statement st = con.createStatement()) { st.executeUpdate("DELETE FROM bus_pass"); }
    }

    public static long insertWithStatement(Connection con, List<String[]> rows)
            throws SQLException {
        clear(con);
        long start = System.nanoTime();
        try (Statement st = con.createStatement()) {
            for (String[] p : rows) {
                String sql = "INSERT INTO bus_pass(pass_id,reg_no,name,route_no,amount) VALUES ('"
                        + p[0] + "','" + p[1] + "','" + p[2] + "'," + p[3] + "," + p[4] + ")";
                st.executeUpdate(sql);
            }
        }
        return (System.nanoTime() - start) / 1_000_000;
    }

    public static long insertWithPrepared(Connection con, List<String[]> rows)
            throws SQLException {
        clear(con);
        long start = System.nanoTime();
        try (PreparedStatement ps = con.prepareStatement(
                "INSERT INTO bus_pass(pass_id,reg_no,name,route_no,amount) VALUES (?,?,?,?,?)")) {
            for (String[] p : rows) {
                bind(ps, p);
                ps.executeUpdate();
```

```
            }
        }
        return (System.nanoTime() - start) / 1_000_000;
    }

    public static long insertWithBatch(Connection con, List<String[]> rows)
            throws SQLException {
        clear(con);
        long start = System.nanoTime();
        PassLoader.loadAll(con, rows);
        return (System.nanoTime() - start) / 1_000_000;
    }

    private static void bind(PreparedStatement ps, String[] p) throws SQLException {
        ps.setString(1, p[0]); ps.setString(2, p[1]); ps.setString(3, p[2]);
        ps.setInt(4, Integer.parseInt(p[3])); ps.setDouble(5, Double.parseDouble(p[4]));
    }

    public static void main(String[] args) throws Exception {
        String url = System.getenv().getOrDefault("DB_URL",
                "jdbc:mysql://localhost:3306/assignment5");
        String user = System.getenv().getOrDefault("DB_USER", "root");
        String password = System.getenv().getOrDefault("DB_PASSWORD", "");
        try (Connection con = DriverManager.getConnection(url, user, password)) {
            for (int size : new int[]{10, 1000, 10000, 100000}) {
                List<String[]> rows = makeRows(size);
                System.out.println("ROWS=" + size);
                System.out.println("Statement ms=" + insertWithStatement(con, rows));
                System.out.println("PreparedStatement ms=" + insertWithPrepared(con, rows));
                System.out.println("Batch ms=" + insertWithBatch(con, rows));
            }
        }
    }
}
```

Task06_Migration/PassMigration.java

```
import java.io.BufferedReader;
import java.nio.file.Files;
import java.nio.file.Path;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

public class PassMigration {
    public static void main(String[] args) throws Exception {
        String url = System.getenv().getOrDefault("DB_URL",
                "jdbc:mysql://localhost:3306/assignment5");
        String user = System.getenv().getOrDefault("DB_USER", "root");
        String password = System.getenv().getOrDefault("DB_PASSWORD", "");
        Path input = Path.of(args.length > 0 ? args[0] : "passes.csv");
        Path failures = Path.of("migration-failures.txt");
        List<String[]> valid = new ArrayList<>();

        try (BufferedReader reader = Files.newBufferedReader(input);
             var writer = Files.newBufferedWriter(failures)) {
            String line;
            int lineNo = 0;
            while ((line = reader.readLine()) != null) {
                lineNo++;
                String[] p = line.split(",", -1);
                try {
                    if (p.length != 5) throw new IllegalArgumentException("Expected 5 fields");
                    Integer.parseInt(p[3].trim());
                    double amount = Double.parseDouble(p[4].trim());
                    if (amount < 0) throw new IllegalArgumentException("Negative amount");
                    valid.add(p);
                } catch (RuntimeException ex) {
                    writer.write("Line " + lineNo + ": " + ex.getMessage() + " | " + line);
                    writer.newLine();
                }
            }
        }

        try (Connection con = DriverManager.getConnection(url, user, password)) {
            int loaded = MigrationBatchLoader.loadAll(con, valid);
            System.out.println("Valid records loaded: " + loaded);
            System.out.println("Rejected records: " + Files.readAllLines(failures).size());
            System.out.println("Failure report: " + failures.toAbsolutePath());
        }
    }
}

class MigrationBatchLoader {
```

```
    static int loadAll(Connection con, List<String[]> passes) throws SQLException {
        String sql = "INSERT INTO bus_pass(pass_id, reg_no, name, route_no, amount) VALUES (?,?,?,?,?)";
        boolean oldAutoCommit = con.getAutoCommit();
        con.setAutoCommit(false);
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            int count = 0;
            for (String[] p : passes) {
                ps.setString(1, p[0]);
                ps.setString(2, p[1]);
                ps.setString(3, p[2]);
                ps.setInt(4, Integer.parseInt(p[3]));
                ps.setDouble(5, Double.parseDouble(p[4]));
                ps.addBatch();
                count++;
                if (count % 1000 == 0) ps.executeBatch();
            }
            ps.executeBatch();
            con.commit();
            return count;
        } catch (SQLException | RuntimeException ex) {
            con.rollback();
            throw ex;
        } finally {
            con.setAutoCommit(oldAutoCommit);
        }
    }
}
```

Task06_Migration/DatabaseInspector.java

```
import java.sql.*;

public class DatabaseInspector {
    public static void main(String[] args) throws Exception {
        String url = System.getenv().getOrDefault("DB_URL",
                "jdbc:mysql://localhost:3306/assignment5");
        String user = System.getenv().getOrDefault("DB_USER", "root");
        String password = System.getenv().getOrDefault("DB_PASSWORD", "");
        try (Connection con = DriverManager.getConnection(url, user, password)) {
            DatabaseMetaData md = con.getMetaData();
            System.out.println("Database: " + md.getDatabaseProductName());
            System.out.println("Database version: " + md.getDatabaseProductVersion());
            System.out.println("Driver: " + md.getDriverName());
            System.out.println("Driver version: " + md.getDriverVersion());
            try (ResultSet tables = md.getTables(con.getCatalog(), null, "%",
                    new String[]{"TABLE"})) {
                while (tables.next())
                    System.out.println("Table: " + tables.getString("TABLE_NAME"));
            }
        }
    }
}
```

Task05_JdbcBatch/MetadataAndColumnCheck.java

```
import java.sql.*;

public class MetadataAndColumnCheck {
    public static void inspectBusPass(Connection con) throws SQLException {
        String sql = "SELECT * FROM bus_pass";
        try (Statement st = con.createStatement(); ResultSet rs = st.executeQuery(sql)) {
            ResultSetMetaData md = rs.getMetaData();
            for (int i = 1; i <= md.getColumnCount(); i++) {
                System.out.println(md.getColumnName(i) + " | " + md.getColumnTypeName(i)
                        + " | " + md.getColumnDisplaySize(i));
            }
        }
    }
}
```

Task01_FileStore/FileStoreExperiment.java

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Arrays;

public class FileStoreExperiment {
    public static void main(String[] args) throws Exception {
        for (int count : new int[]{1000, 10000, 50000, 100000}) {
            Path file = Files.createTempFile("pass-growth", ".csv");
            PassStore store = new PassStore(file);
            for (int i = 1; i <= count; i++)
                store.add("P" + i, "REG" + i, "Student" + i, i % 25, 1000 + i);

            long start = System.nanoTime();
            String[] found = store.find("P" + count);
            long elapsedMs = (System.nanoTime() - start) / 1_000_000;
```

```
            System.out.println("Records=" + count + ", found=" +
                    (found != null && found[0].equals("P" + count)) +
                    ", search_ms=" + elapsedMs);
            Files.deleteIfExists(file);
        }
    }
}
```

Task01_FileStore/LineCounterBenchmark.java

```
import java.nio.file.Files;
import java.nio.file.Path;

public class LineCounterBenchmark {
    public static void main(String[] args) throws Exception {
        for (int sizeMb : new int[]{1, 10, 50}) {
            Path file = Files.createTempFile("attendance-" + sizeMb + "mb", ".log");
            byte[] block = new byte[1024 * 1024];
            java.util.Arrays.fill(block, (byte)'A');
            java.util.Arrays.fill(block, block.length - 1, block.length, (byte)'\n');
            try (var out = Files.newOutputStream(file)) {
                for (int i = 0; i < sizeMb; i++) out.write(block);
            }

            long start = System.nanoTime();
            long unbuffered = BufferedLineCounter.countUnbuffered(file);
            long unbufferedMs = (System.nanoTime() - start) / 1_000_000;

            start = System.nanoTime();
            long buffered = BufferedLineCounter.countBuffered(file);
            long bufferedMs = (System.nanoTime() - start) / 1_000_000;

            System.out.println("Size MB=" + sizeMb + ", lines=" + unbuffered +
                    ", unbuffered_ms=" + unbufferedMs + ", buffered_ms=" + bufferedMs);
            if (unbuffered != buffered) throw new IllegalStateException("Count mismatch");
            Files.deleteIfExists(file);
        }
    }
}
```

Task04_Transactions/WalletTransferDemo.java

```
import java.math.BigDecimal;
import java.sql.*;

public class WalletTransferDemo {
    public static void main(String[] args) throws Exception {
        String url = System.getenv().getOrDefault("DB_URL",
                "jdbc:mysql://localhost:3306/assignment5");
        String user = System.getenv().getOrDefault("DB_USER", "root");
        String password = System.getenv().getOrDefault("DB_PASSWORD", "");
        try (Connection con = DriverManager.getConnection(url, user, password)) {
            WalletTransfer.transfer(con, 1, 2, new BigDecimal("500.00"));
            try (Statement st = con.createStatement();
                 ResultSet rs = st.executeQuery(
                         "SELECT wallet_id, balance FROM wallet ORDER BY wallet_id")) {
                while (rs.next())
                    System.out.println(rs.getInt(1) + " -> " + rs.getBigDecimal(2));
            }
        }
    }
}
```

Task05_JdbcBatch/schema.sql

```
CREATE DATABASE IF NOT EXISTS assignment5;
USE assignment5;

CREATE TABLE IF NOT EXISTS bus_pass (
    pass_id VARCHAR(20) PRIMARY KEY,
    reg_no VARCHAR(20) NOT NULL,
    name VARCHAR(100) NOT NULL,
    route_no INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL
);

CREATE TABLE IF NOT EXISTS student (
    student_id INT PRIMARY KEY,
    reg_no VARCHAR(20) NOT NULL UNIQUE,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(120) NOT NULL
);

CREATE TABLE IF NOT EXISTS wallet (
    wallet_id INT PRIMARY KEY,
    balance DECIMAL(12,2) NOT NULL CHECK (balance >= 0)
);
```

```
CREATE TABLE IF NOT EXISTS app_user (
    user_id VARCHAR(40) PRIMARY KEY,
    user_pwd VARCHAR(100) NOT NULL
);

INSERT INTO wallet(wallet_id, balance) VALUES
(1, 5000.00), (2, 1000.00)
ON DUPLICATE KEY UPDATE balance = VALUES(balance);

INSERT INTO app_user(user_id, user_pwd) VALUES
('student1', 'safePass123')
ON DUPLICATE KEY UPDATE user_pwd = VALUES(user_pwd);
```

Task06_Migration/passes.csv

```
P1001,192311001,Arun,12,4500.00
P1002,192311002,Divya,7,3800.00
P1003,192311003,Kavin,12,4200.00
BAD-LINE,ONLY-TWO
P1004,192311004,Meena,abc,3000.00
P1005,192311005,Ravi,3,2500.00
```

APPENDIX B – SQL DATABASE SCHEMA

```
CREATE DATABASE IF NOT EXISTS assignment5;
USE assignment5;

CREATE TABLE IF NOT EXISTS bus_pass (
    pass_id VARCHAR(20) PRIMARY KEY,
    reg_no VARCHAR(20) NOT NULL,
    name VARCHAR(100) NOT NULL,
    route_no INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL
);

CREATE TABLE IF NOT EXISTS student (
    student_id INT PRIMARY KEY,
    reg_no VARCHAR(20) NOT NULL UNIQUE,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(120) NOT NULL
);

CREATE TABLE IF NOT EXISTS wallet (
    wallet_id INT PRIMARY KEY,
    balance DECIMAL(12,2) NOT NULL CHECK (balance >= 0)
);

CREATE TABLE IF NOT EXISTS app_user (
    user_id VARCHAR(40) PRIMARY KEY,
    user_pwd VARCHAR(100) NOT NULL
);

INSERT INTO wallet(wallet_id, balance) VALUES
(1, 5000.00), (2, 1000.00)
ON DUPLICATE KEY UPDATE balance = VALUES(balance);

INSERT INTO app_user(user_id, user_pwd) VALUES
('student1', 'safePass123')
ON DUPLICATE KEY UPDATE user_pwd = VALUES(user_pwd);
```

APPENDIX C – SAMPLE MIGRATION INPUT

```
P1001,192311001,Arun,12,4500.00
P1002,192311002,Divya,7,3800.00
P1003,192311003,Kavin,12,4200.00
BAD-LINE,ONLY-TWO
P1004,192311004,Meena,abc,3000.00
P1005,192311005,Ravi,3,2500.00
```